1-DP-Playing with Numbers

Started on	Thursday, 8 October 2026, 10:09 AM
State	Finished
Completed on	Thursday, 8 October 2026, 10:11 AM
Time taken	1 min 58 secs
Grade	**10.00** out of 10.00 (**100**%)

Question 1 Correct Mark 10.00 out of 10.00 Flag question

Playing with Numbers:

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

Example 1:

Input: *6*

Output:*6*

Explanation: *There are 6 ways to 6 represent number with 1 and 3*

1+1+1+1+1+1

3+3

1+1+1+3

1+1+3+1

1+3+1+1

3+1+1+1

Input Format

First Line contains the number n

Output Format

Print: The number of possible ways 'n' can be represented using 1 and 3

Sample Input

6

Sample Output

6

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

long long countWays(int n) {
    if (n < 0) return 0;
    if (n == 0 || n == 1 || n == 2) return 1;

    long long dp[n + 1];
    dp[0] = 1;
    dp[1] = 1;
    dp[2] = 1;

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 3];
    }

    return dp[n];
```

```
    }

    return dp[n];
}

int main() {
    int n;
    if (scanf("%d", &n) == 1) {
        printf("%lld\n", countWays(n));
    }
    return 0;
}
```

	Input	Expected	Got	
✔	6	6	6	✔
✔	25	8641	8641	✔
✔	100	24382819596721629	24382819596721629	✔

Passed all tests! ✔

2-DP-Playing with chessboard

Started on	Thursday, 8 October 2026, 10:11 AM
State	Finished
Completed on	Thursday, 8 October 2026, 10:21 AM
Time taken	10 mins 10 secs
Grade	**10.00** out of 10.00 (**100%**)

Question 1 Correct Mark 10.00 out of 10.00 Flag question

Playing with Chessboard:

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been gi
black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can onl
the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3

1 2 4

2 3 4

8 7 1

Output:

19

Explanation:

Totally there will be 6 paths among that the optimal is

Output Format

Print Maximum monetary value of the path

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

long long getMaxMonetaryPath(int n, long long grid[n][n]) {
    long long dp[n][n];

    dp[0][0] = grid[0][0];

    for (int j = 1; j < n; j++) {
        dp[0][j] = dp[0][j - 1] + grid[0][j];
    }

    for (int i = 1; i < n; i++) {
        dp[i][0] = dp[i - 1][0] + grid[i][0];
    }

    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            long long maxPrev = (dp[i - 1][j] > dp[i][j - 1]) ? dp[i - 1][j] : dp[i][j - 1];
            dp[i][j] = grid[i][j] + maxPrev;
        }
    }

    return dp[n - 1][n - 1];
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    long long grid[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%lld", &grid[i][j]);
```

```
38      printf("%lld\n", getMaxMonetaryPath(n, grid));
39
40      return 0;
41  }
```

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

3-DP-Longest Common Subsequence

Started on	Thursday, 8 October 2026, 10:21 AM
State	Finished
Completed on	Thursday, 8 October 2026, 10:28 AM
Time taken	6 mins 30 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 Correct Mark 1.00 out of 1.00 Flag question

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1	a	g	**g**	**t**	**a**	**b**	
s2	**g**	x	**t**	**x**	**a**	**y**	**b**

Input	Result
aab azb	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int getLCSLength(char* s1, char* s2) {
    int m = strlen(s1);
    int n = strlen(s2);
    int dp[m + 1][n + 1];

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            } else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    return dp[m][n];
}

int main() {
    char s1[1000], s2[1000];
    if (scanf("%s %s", s1, s2) == 2) {
        printf("%d\n", getLCSLength(s1, s2));
    }
    return 0;
}
```

```
    return dp[m][n];
}

int main() {
    char s1[1000], s2[1000];
    if (scanf("%s %s", s1, s2) == 2) {
        printf("%d\n", getLCSLength(s1, s2));
    }
    return 0;
}
```

	Input	Expected	Got	
✔	aab azb	2	2	✔
✔	ABCD ABCD	4	4	✔

Passed all tests! ✔

4-DP-Longest non-decreasing Subsequence

Started on	Thursday, 8 October 2026, 10:22 AM
State	Finished
Completed on	Thursday, 8 October 2026, 10:28 AM
Time taken	5 mins 55 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int longestNonDecreasingSubsequence(int n, int arr[]) {
    if (n == 0) return 0;

    int dp[n];
    int maxLen = 1;

    for (int i = 0; i < n; i++) {
        dp[i] = 1;
        for (int j = 0; j < i; j++) {
            if (arr[i] >= arr[j]) {
                if (dp[j] + 1 > dp[i]) {
                    dp[i] = dp[j] + 1;
                }
            }
        }
        if (dp[i] > maxLen) {
            maxLen = dp[i];
        }
    }
    return maxLen;
}

int main() {
    int n;
    if (scanf("%d", &n) == 1) {
        int arr[n];
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }
        printf("%d\n", longestNonDecreasingSubsequence(n, arr));
    }
    return 0;
}
```

	Input	Expected	Got	
✔	9 -1 3 4 5 2 2 2 2 3	6	6	✔
✔	7 1 2 2 4 5 7 6	6	6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.